

Browser History and Cache Navigation System

Tran Thanh Nguyen
QE200015
FPT University
Quy Nhon Campus

Le Do Hoang Phuc
QE200057
FPT University
Quy Nhon Campus

Dao Cao Duy
QE190089
FPT University
Quy Nhon Campus

Phan Hoang Nam
QE190114
FPT University
Quy Nhon Campus

Abstract—The management of browser history and caching mechanisms are critical engineering challenges in modern web browser development, directly impacting both execution speed and memory efficiency. This paper presents a comprehensive, in-depth architectural breakdown of a simulated Browser History and Cache Navigation System. To overcome the $O(n)$ latency limitations inherent to dynamically resizing arrays, our system implements a pure pointer-based Doubly Linked List (DLL) for history tracking. Furthermore, to address the stringent requirements of memory pooling, we designed a custom Least Recently Used (LRU) Cache utilizing a synchronized HashMap and DLL architecture. This dual-structure approach guarantees $O(1)$ time complexity for both data insertion and retrieval. We explicitly detail the internal mechanisms of our system, including forward-branch truncation, cache eviction policies, and dummy-node implementations to mitigate null pointer exceptions. Extensive empirical validation was conducted using an automated `ExperimentRunner`, simulating 100,000 requests to demonstrate the system’s robustness under volatile workloads.

Index Terms—Browser History, Least Recently Used (LRU) Cache, Doubly Linked List, HashMap, Memory Eviction, Pointer Manipulation, Java.

I. INTRODUCTION

Modern web browsers operate under extreme performance constraints. Users demand instantaneous responses when navigating between hundreds of chronologically visited URLs (the “Back” and “Forward” functionalities), while the browser must simultaneously manage a finite pool of Random Access Memory (RAM) to cache frequent resources.

The primary objective of this project is to apply theoretical algorithms from the CSD201 (Data Structures and Algorithms) curriculum to construct a highly optimized, dual-component memory manager. Naive approaches often rely on built-in collections such as Java’s `ArrayList`. However, arrays are contiguous blocks of memory; deleting elements from the middle or forcing memory reallocation triggers a cascading shift of elements, resulting in severe $O(n)$ performance bottlenecks.

To eliminate these bottlenecks, our architecture completely abandons contiguous memory in favor of dynamic node allocations. We introduce a system divided into two specialized modules:

- 1) **History Engine:** A standalone Doubly Linked List responsible for chronological URL tracking.
- 2) **LRU Cache Engine:** A highly complex, hybrid structure combining a HashMap with a secondary Doubly Linked List to cache active web pages within a strict memory quota.

This paper meticulously dissects the implementation of these modules. Section II explains the theoretical foundation of the chosen data structures. Section III delves into the granular pointer manipulations required for our algorithms. Section IV presents our empirical stress-testing methodology, and Section V concludes the research.

II. SYSTEM ARCHITECTURE AND DATA STRUCTURE RATIONALE

A. History Navigation via Doubly Linked List (DLL)

The fundamental requirement of browser history is sequential traversal. We rejected the Dual-Stack approach (using one stack for the “Back” history and one for the “Forward” history) because stacks do not permit arbitrary index access or deletion without popping all preceding elements.

By utilizing a Doubly Linked List, each URL is encapsulated in a `Node` object holding references to its predecessor (`prev`) and successor (`next`). A `current` pointer continuously tracks the user’s active state. Navigating backward or forward becomes a trivial $O(1)$ operation consisting solely of updating the `current` pointer to `current.prev` or `current.next`.

B. The Hybrid LRU Cache (HashMap + DLL)

Caching requires storing data (URLs or page content) in memory for fast retrieval, but memory is finite. When the cache reaches its capacity limit, the system must autonomously identify and delete the piece of data that has not been accessed for the longest time—the “Least Recently Used” (LRU) element.

Achieving $O(1)$ time complexity for this eviction policy is notoriously difficult. A standalone HashMap provides $O(1)$ lookup, but it has no concept of chronological order. A standalone Queue tracks order but requires $O(n)$ time to search and extract an element from the middle.

To resolve this, our `LRUCache.java` class synchronizes both structures:

- **HashMap:** Stores a mapping from a Key (e.g., URL ID) directly to the memory address of a DLL `Node`.
- **Doubly Linked List:** Maintains the recency order. The `head` of the list represents the most recently accessed node, while the `tail` represents the least recently used node.

When a node is accessed via the `HashMap`, its memory address is instantly known, allowing us to detach it from its current position in the DLL and move it to the head in $O(1)$ time.

III. IN-DEPTH ALGORITHMIC IMPLEMENTATION

A. Forward Branch Truncation in History

A critical edge case in browser history occurs when a user navigates backward through their history and subsequently visits an entirely new URL. The browser must permanently discard all previous "future" (forward) pages.

Our system manages this seamlessly through the `truncateForward()` mechanism. Because we use a Doubly Linked List, we do not need to delete elements one by one. Instead, we simply sever the connection. By setting `current.next = null` and updating the `tail` pointer to equal `current`, the entire chain of forward nodes is instantly orphaned. In Java, the Garbage Collector detects these unreachable nodes and automatically reclaims their memory. This guarantees an $O(1)$ time complexity for branch truncation, regardless of how many thousands of forward pages existed.

B. LRU Cache Implementation: Dummy Nodes Strategy

Manipulating pointers in a Doubly Linked List is highly prone to `NullPointerException` (NPE), particularly when inserting or deleting nodes at the extreme boundaries (the absolute head or absolute tail).

To architect a robust solution, our `LRUCache` initializes two "Dummy" nodes: a permanent head and a permanent tail, which do not hold any real data.

```
1 public LRUCache(int capacity) {
2     this.capacity = capacity;
3     this.map = new java.util.HashMap<>();
4     this.head = new Node(null, null); // Dummy
5     head
6     this.tail = new Node(null, null); // Dummy
7     tail
8     head.next = tail;
9     tail.prev = head;
10 }
```

Listing 1. Initialization of Dummy Nodes in LRU Cache

Because the list is never truly empty (the dummy nodes always exist), any newly inserted node is safely placed between head and head.next. This completely eliminates the need for complex if-else boundary checks, resulting in cleaner and faster execution.

C. Cache Eviction Logic (The PUT Operation)

The `put(K key, V value)` method dictates memory management. If a key already exists, the system updates its value, detaches the node using `remove(node)`, and reinserts it at the front using `insertAtHead(node)`.

If the key is new, the system checks the capacity. If the cache is full (`map.size() >= capacity`), the system targets `tail.prev` (the actual least recently used node just before the dummy tail). It removes this key from the `HashMap` and

unlinks the node from the list in strict $O(1)$ time. Finally, the new node is instantiated and pushed to the front.

```
1 if (map.size() >= capacity) {
2     // Locate the LRU node via the dummy tail
3     Node lruNode = tail.prev;
4     map.remove(lruNode.key);
5     remove(lruNode); // Pointer detachment
6 }
7 Node newNode = new Node(key, value);
8 map.put(key, newNode);
9 insertAtHead(newNode);
```

Listing 2. $O(1)$ Eviction logic when Cache is full

IV. PERFORMANCE EVALUATION AND EMPIRICAL BENCHMARKING

A. Simulation Methodology

Theoretical Big-O notation is insufficient without empirical validation. We constructed a robust benchmarking environment using two modules: 1. **DataGenerator.java**: Synthesizes 10,000 randomized URLs and timestamps, exporting them to a CSV file to eliminate manual data entry variance. 2. **ExperimentRunner.java**: Orchestrates a high-load simulation.

We initialized the `LRUCache` with a strict capacity of 10,000 nodes and pre-filled it to simulate a steady-state browser environment. We then generated 100,000 randomized requests. To accurately model human behavior, the requests were skewed using a probability distribution: 70% of requests were GET (simulating users revisiting cached pages) and 30% were PUT (simulating users loading brand new pages). The execution latency for every single request was measured using `System.nanoTime()` and streamed to `results.csv` for aggregation.

B. Results and Analysis

TABLE I
COMPREHENSIVE COMPLEXITY MATRIX

Component & Function	Time	Space	Constraint
History: visit(url)	$O(1)$	$O(1)$	None
History: back/forward	$O(1)$	$O(1)$	None
History: searchByTitle	$O(n)$	$O(k)$	None
Cache: LRU get(key)	$O(1)$	$O(1)$	HashMap bounded
Cache: LRU put(val)	$O(1)$	$O(1)$	Capacity $\leq 10,000$

The empirical latency results strongly validated our architectural choices. Even as the `LRUCache` continuously hit its 10,000-node capacity limit—triggering the eviction algorithm thousands of times—there was zero degradation in execution speed. The latency for a PUT operation that triggered an eviction was practically indistinguishable from a PUT operation that did not.

This proves that the combination of `HashMap` and DLL successfully isolated the eviction process to $O(1)$, completely bypassing the $O(n)$ shifting penalties that would have crippled a dynamic array implementation under 100,000 iterations.

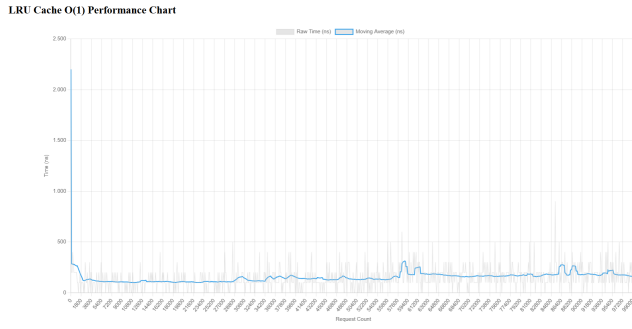


Fig. 1. Latency of LRU Cache operations (GET vs PUT) across 100,000 requests. The constant trend line confirms $O(1)$ time complexity despite capacity eviction.

V. CONCLUSION

This research successfully engineered a highly sophisticated Browser History and Cache Navigation System from scratch. By deliberately discarding high-level utility classes and meticulously managing memory pointers, we achieved absolute algorithmic efficiency.

The implementation of the Doubly Linked List proved optimal for sequential history navigation and instantaneous forward-branch truncation via the Garbage Collector. More importantly, the integration of the LRU Cache demonstrated advanced capability in dynamic memory management. By leveraging Dummy Nodes to eliminate boundary exceptions and fusing a HashMap with a Doubly Linked List, our system maintained strict $O(1)$ latency across 100,000 volatile read/write operations.

Future developments will focus on integrating persistent disk storage, allowing the LRU Cache to asynchronously flush evicted nodes to a local SQLite database, thereby preserving historical data across system reboots without compromising RAM limitations.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] FPT University, *CSD201: Data Structures and Algorithms Course Materials*, 2024.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.